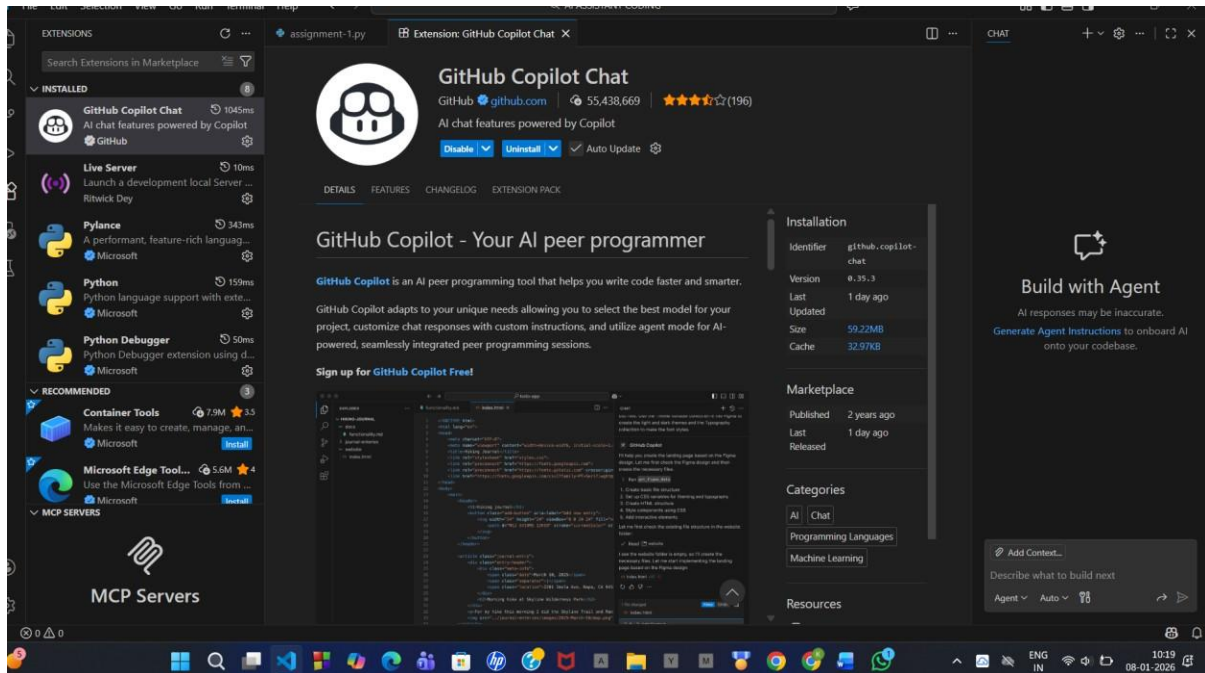


AI-ASSISTED CODING

ASSIGNMENT-1

2303A51242 BATCH-05

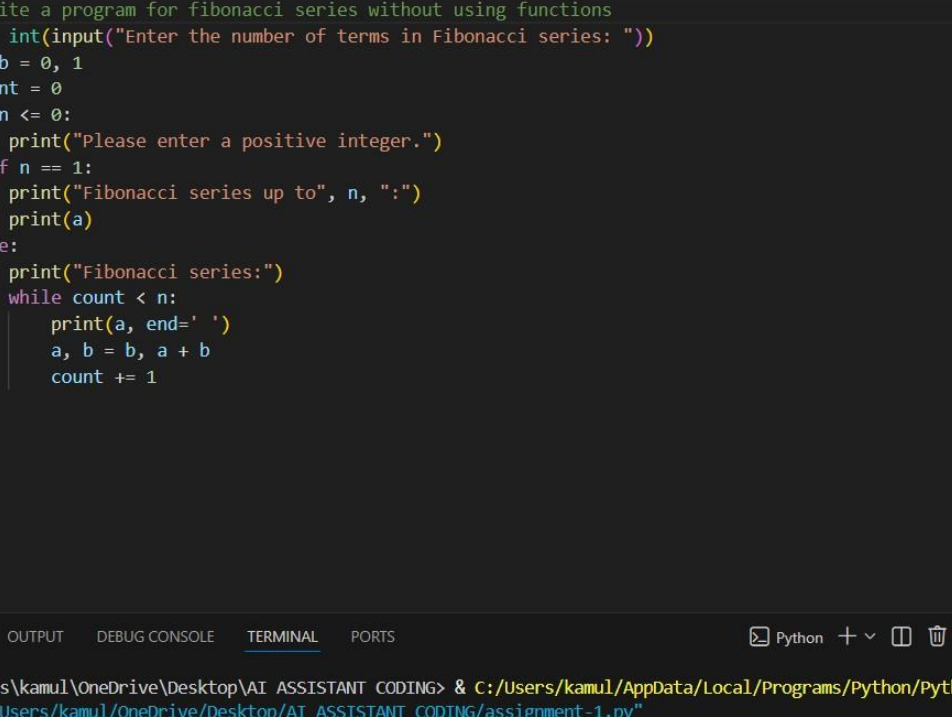
TASK-0:



TASK-01:

#write a program for fibonacci series without using functions

```
assignment-1.py X
assignment-1.py > ...
1 #write a program for fibonacci series without using functions
2 n = int(input("Enter the number of terms in Fibonacci series: "))
3 a, b = 0, 1
4 count = 0
5 if n <= 0:
6     print("Please enter a positive integer.")
7 elif n == 1:
8     print("Fibonacci series up to", n, ":")
9     print(a)
10 else:
11     print("Fibonacci series:")
12     while count < n:
13         print(a, end=' ')
14         a, b = b, a + b
15         count += 1
16
```



The image shows a Visual Studio Code editor window with a file named 'assignment-1.py'. The code is a Python script that generates a Fibonacci series. It prompts the user to enter the number of terms, checks for valid input, and then prints the series. The terminal at the bottom shows the script being executed, with the user entering '10' and the output displaying the first 10 terms of the Fibonacci series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

```
1 #write a program for fibonacci series without using functions
2 n = int(input("Enter the number of terms in Fibonacci series: "))
3 a, b = 0, 1
4 count = 0
5 if n <= 0:
6     print("Please enter a positive integer.")
7 elif n == 1:
8     print("Fibonacci series up to", n, ":")
9     print(a)
10 else:
11     print("Fibonacci series:")
12     while count < n:
13         print(a, end=' ')
14         a, b = b, a + b
15         count += 1
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + - [] [X] ... [] [X] [X]

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python
n.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-1.py"
Enter the number of terms in Fibonacci series: 10
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

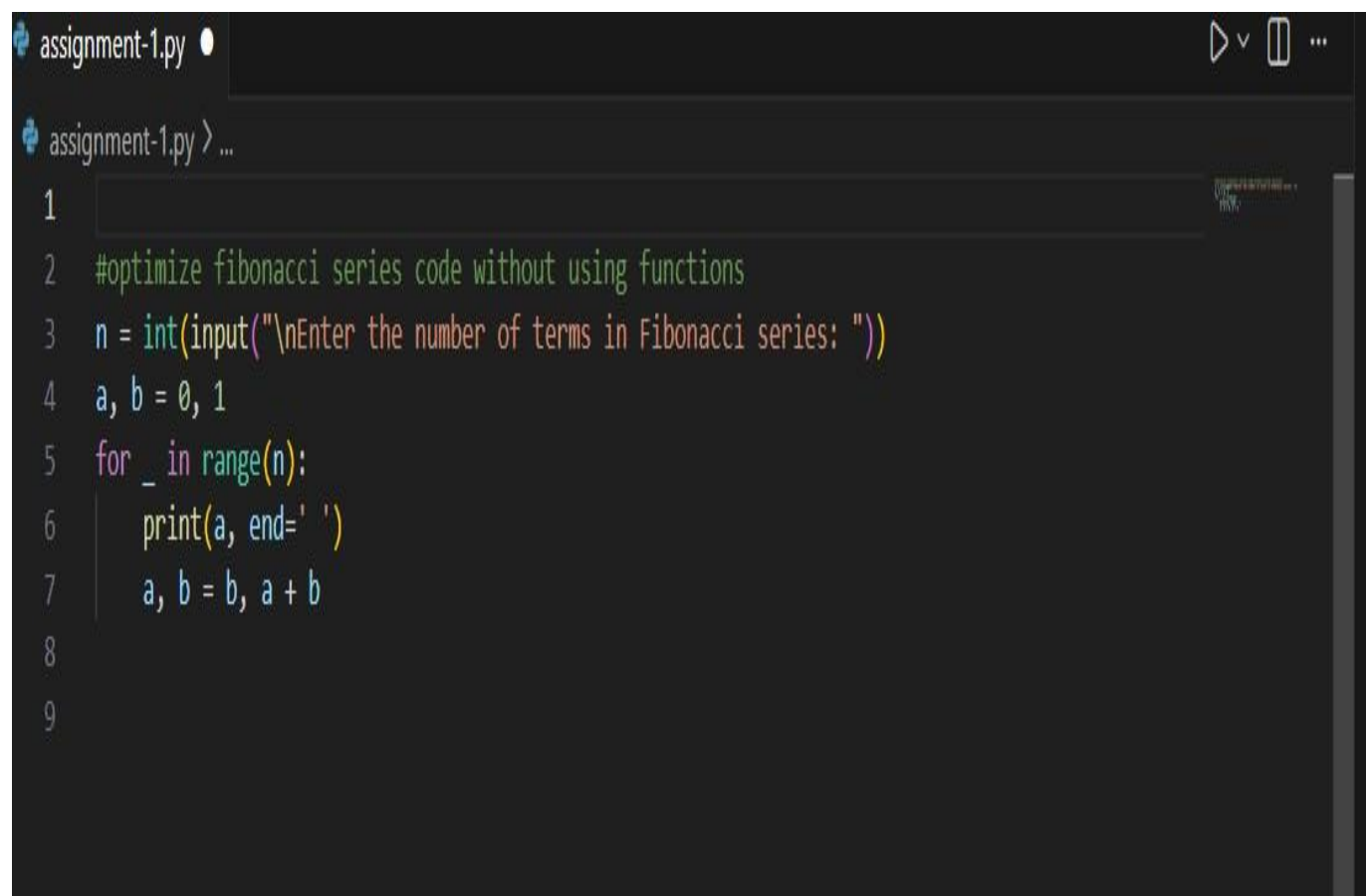
EXPLANATION:

This program generates the Fibonacci series without using any functions by using variables, conditional statements, and a while loop. The user enters the

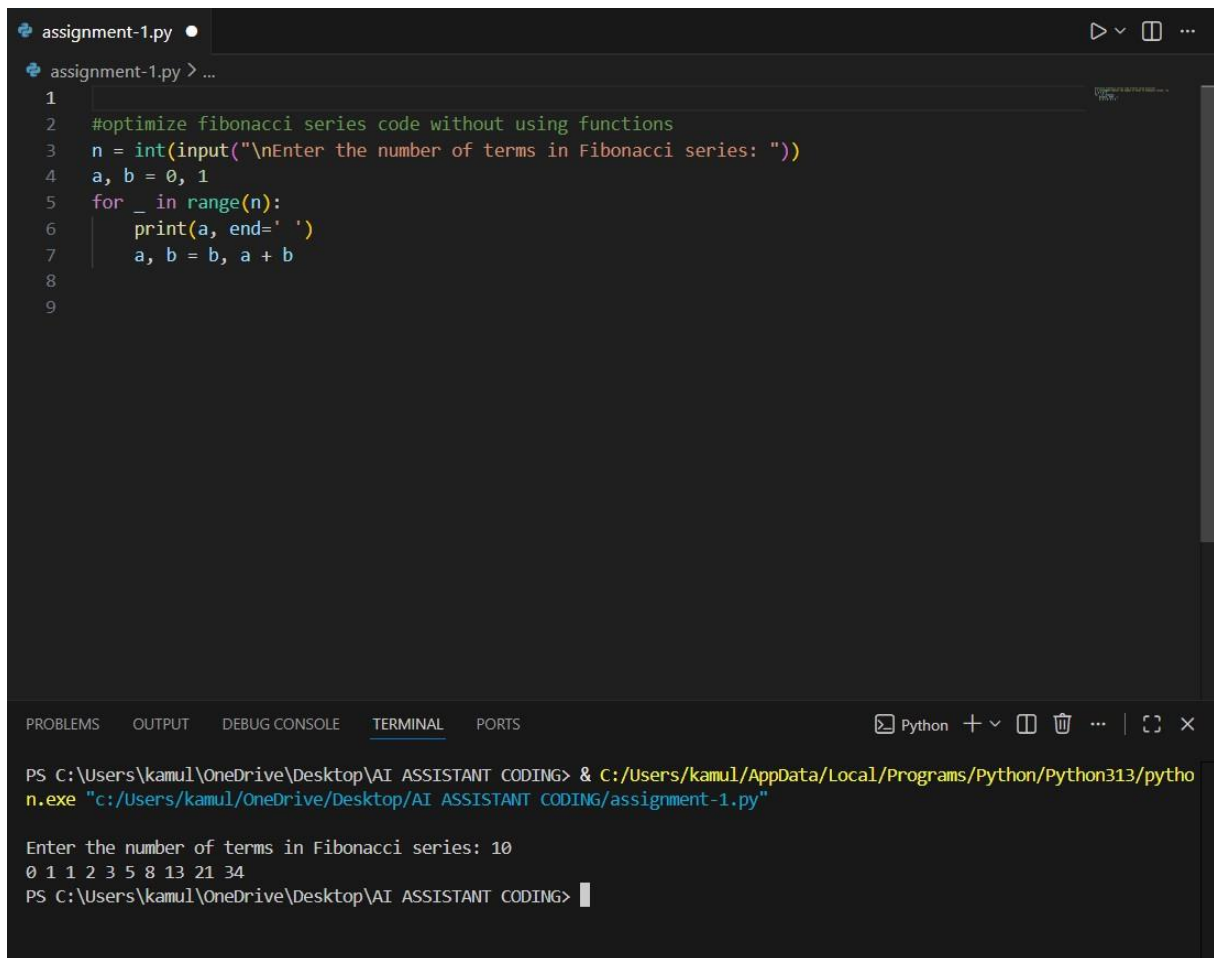
number of terms to be printed. Two variables a and b are initialized with values 0 and 1 to represent the first two Fibonacci numbers. If the user enters a non-positive number, the program displays an error message. If the input is valid, the program uses a while loop to repeatedly print the current Fibonacci number and update the next value by adding the previous two numbers. This approach demonstrates how the Fibonacci series can be generated iteratively using basic programming constructs.

TASK-02:

#optimize fibonacci series code without using functions

A screenshot of a code editor window with a dark theme. The title bar at the top shows 'assignment-1.py' and some window control icons. The editor area contains a Python script. Line 1 is empty. Line 2 is a comment: '#optimize fibonacci series code without using functions'. Line 3 is 'n = int(input("\nEnter the number of terms in Fibonacci series: "))'. Line 4 is 'a, b = 0, 1'. Line 5 is 'for _ in range(n):'. Line 6 is ' print(a, end=' ')'. Line 7 is ' a, b = b, a + b'. Line 8 is empty. Line 9 is empty. The code is color-coded: comments are green, strings are red, integers are blue, and keywords are purple. A vertical scrollbar is visible on the right side of the editor.

```
1  
2 #optimize fibonacci series code without using functions  
3 n = int(input("\nEnter the number of terms in Fibonacci series: "))  
4 a, b = 0, 1  
5 for _ in range(n):  
6     print(a, end=' ' )  
7     a, b = b, a + b  
8  
9
```



```
assignment-1.py
1
2 #optimize fibonacci series code without using functions
3 n = int(input("\nEnter the number of terms in Fibonacci series: "))
4 a, b = 0, 1
5 for _ in range(n):
6     print(a, end=' ')
7     a, b = b, a + b
8
9

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-1.py"
Enter the number of terms in Fibonacci series: 10
0 1 1 2 3 5 8 13 21 34
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

EXPLANATION:

This program prints the Fibonacci series in an optimized and efficient way without using functions. The user enters the number of terms to be displayed. The variables `a` and `b` are initialized to 0 and 1, representing the first two Fibonacci numbers. A for loop runs `n` times, and in each iteration the current value of `a` is printed. Then, the next Fibonacci number is calculated by updating `a` and `b` as the sum of the previous two numbers. This approach avoids unnecessary condition checks and uses minimal variables, making the program simple and efficient.

fibonacci series code with using functions

```

1
2 # fibonacci series code with using functions
3 def fibonacci(n):
4     a, b = 0, 1
5     series = []
6     for _ in range(n):
7         series.append(a)
8         a, b = b, a + b
9     return series
10 num_terms = 10
11 print(f"Fibonacci series up to {num_terms} terms:")
12 print(fibonacci(num_terms))
13

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-1.py"

Fibonacci series up to 10 terms:
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>

This program generates the Fibonacci series using a function. The function fibonacci(n) initializes the first two Fibonacci numbers as 0 and 1 and uses a for loop to compute the series up to n terms. Each Fibonacci number is stored in a list called series. After calculating all terms, the function returns the list of

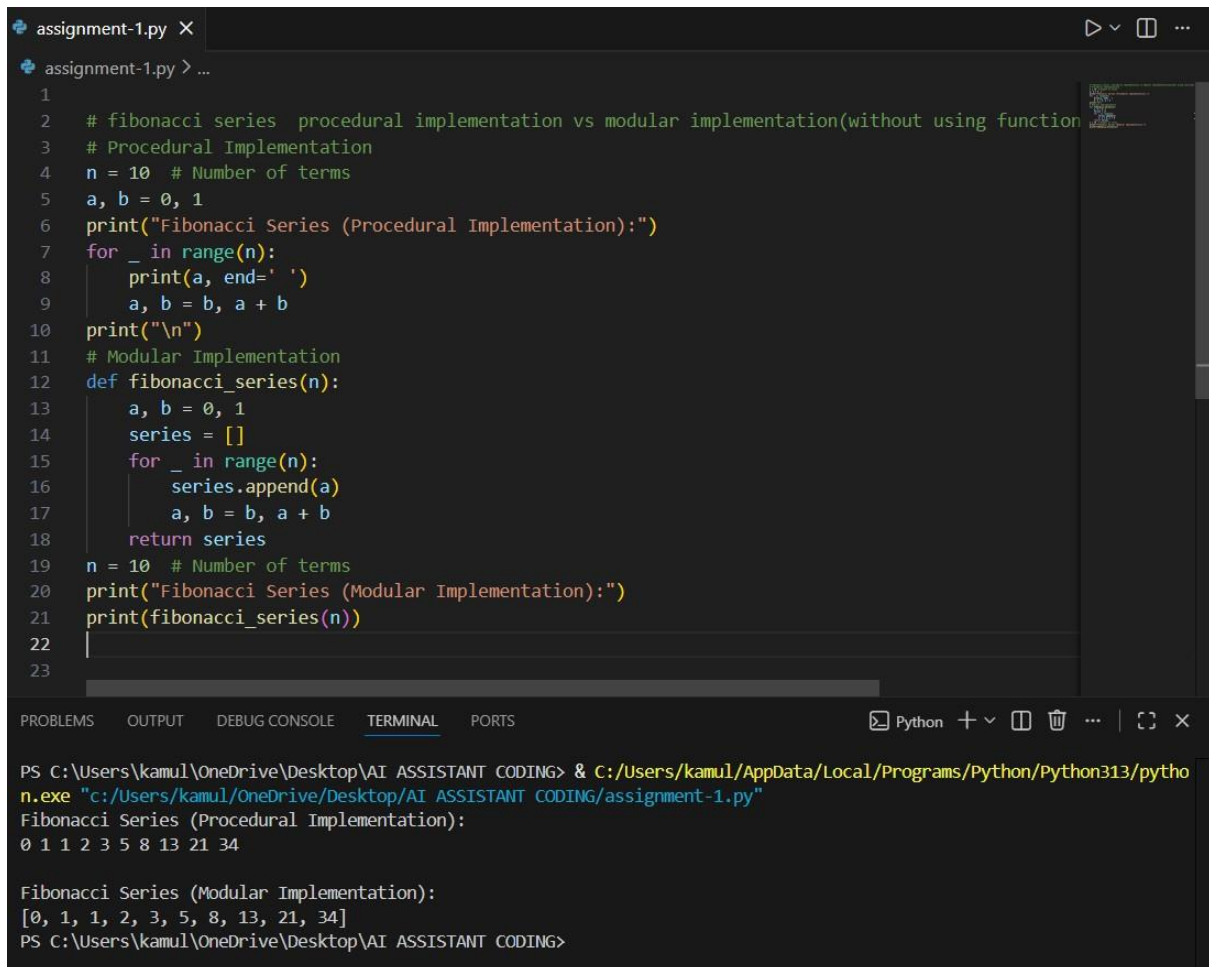
Fibonacci numbers. In the main part of the program, the number of terms is set to 10, and the function is called to display the Fibonacci series. This approach improves code readability, reusability, and organization by using functions.

TASK-04:

fibonacci series procedural implementation vs modular implementation(without using functions vs with functions)

```
assignment-1.py
assignment-1.py > ...

1
2 # fibonacci series procedural implementation vs modular implementation(without using function
3 # Procedural Implementation
4 n = 10 # Number of terms
5 a, b = 0, 1
6 print("Fibonacci Series (Procedural Implementation):")
7 for _ in range(n):
8     print(a, end=' ')
9     a, b = b, a + b
10 print("\n")
11 # Modular Implementation
12 def fibonacci_series(n):
13     a, b = 0, 1
14     series = []
15     for _ in range(n):
16         series.append(a)
17         a, b = b, a + b
18     return series
19 n = 10 # Number of terms
20 print("Fibonacci Series (Modular Implementation):")
21 print(fibonacci_series(n))
22
23
```


A screenshot of a Python IDE window titled 'assignment-1.py'. The code defines two ways to generate the first 10 Fibonacci numbers. The first is a procedural implementation using a loop and variables 'a' and 'b'. The second is a modular implementation using a function 'fibonacci_series(n)' that returns a list. The terminal at the bottom shows the execution of the script, displaying the output for both methods. The output for the procedural method is '0 1 1 2 3 5 8 13 21 34' and for the modular method is '[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]'.

```
1
2 # fibonacci series procedural implementation vs modular implementation(without using function)
3 # Procedural Implementation
4 n = 10 # Number of terms
5 a, b = 0, 1
6 print("Fibonacci Series (Procedural Implementation):")
7 for _ in range(n):
8     print(a, end=' ')
9     a, b = b, a + b
10 print("\n")
11 # Modular Implementation
12 def fibonacci_series(n):
13     a, b = 0, 1
14     series = []
15     for _ in range(n):
16         series.append(a)
17         a, b = b, a + b
18     return series
19 n = 10 # Number of terms
20 print("Fibonacci Series (Modular Implementation):")
21 print(fibonacci_series(n))
22
23
```

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-1.py"

Fibonacci Series (Procedural Implementation):
0 1 1 2 3 5 8 13 21 34

Fibonacci Series (Modular Implementation):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>

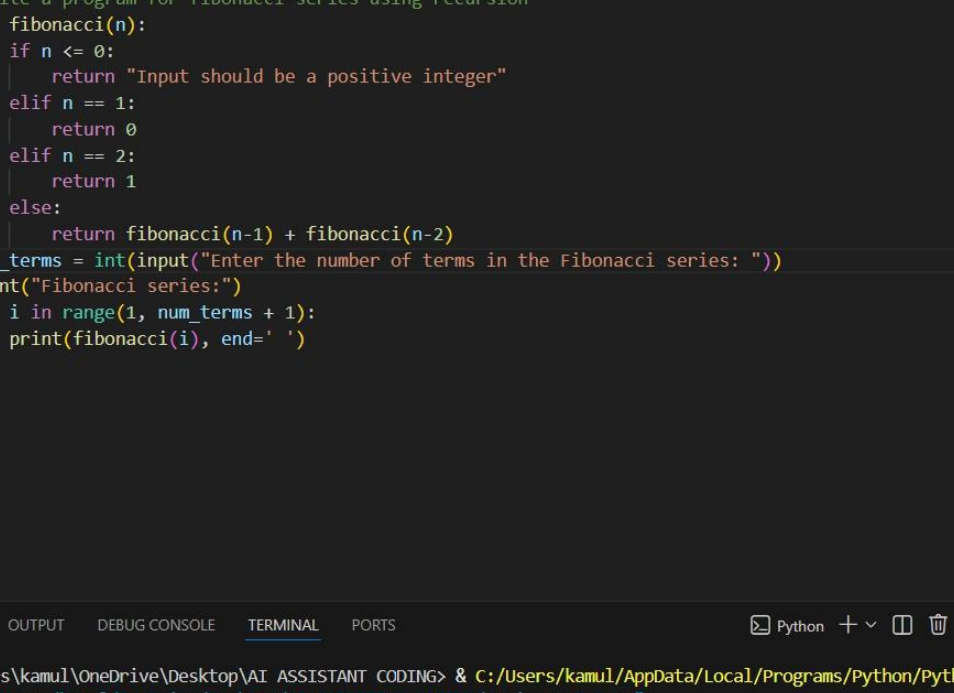
EXPLANATION:

This program demonstrates two different approaches to generating the Fibonacci series: procedural implementation and modular implementation. In the procedural approach, the logic for generating the Fibonacci series is written directly in the main program without using any functions. The variables `a` and `b` are initialized to 0 and 1, and a for loop is used to print the Fibonacci numbers one by one. This method is simple but less reusable. In the modular approach, the Fibonacci logic is placed inside a function called `fibonacci_series(n)`. The function computes the Fibonacci numbers using a loop, stores them in a list, and returns the list. The main program then calls this function to display the series. This approach improves code organization, readability, and reusability. Overall, the program highlights the difference between procedural and modular programming by showing how the same problem can be solved in two structured ways.

TASK-05:

#write a program for fibonacci series using recursion

```
assignment-1.py X
assignment-1.py > ...
1 #write a program for fibonacci series using recursion
2 def fibonacci(n):
3     if n <= 0:
4         return "Input should be a positive integer"
5     elif n == 1:
6         return 0
7     elif n == 2:
8         return 1
9     else:
10        return fibonacci(n-1) + fibonacci(n-2)
11 num_terms = int(input("Enter the number of terms in the Fibonacci series: "))
12 print("Fibonacci series:")
13 for i in range(1, num_terms + 1):
14     print(fibonacci(i), end=' ')
15
```



```
assignment-1.py X
assignment-1.py > ...
1 #write a program for fibonacci series using recursion
2 def fibonacci(n):
3     if n <= 0:
4         return "Input should be a positive integer"
5     elif n == 1:
6         return 0
7     elif n == 2:
8         return 1
9     else:
10        return fibonacci(n-1) + fibonacci(n-2)
11 num_terms = int(input("Enter the number of terms in the Fibonacci series: "))
12 print("Fibonacci series:")
13 for i in range(1, num_terms + 1):
14     print(fibonacci(i), end= ' ')
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + - [] [X] ... | [] [X]

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-1.py"
Enter the number of terms in the Fibonacci series: 10
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
```

EXPLANATION:

This program generates the Fibonacci series using a recursive function. The function fibonacci(n) returns the n -th Fibonacci number. Base conditions are used to return 0 when $n = 1$ and 1 when $n = 2$, ensuring the recursion stops. For values greater than 2, the function calls itself to calculate the sum of the

previous two Fibonacci numbers. The user enters the number of terms, and a loop prints the Fibonacci series up to that number. This program demonstrates the concept of recursion by solving a problem through repeated function calls.